What v2 proves, what improved, and what it costs (for the review)

This page is written to help you explain the project to your mentor: what it is trying to prove, what got better compared with v1, what got worse, how the results were checked, and what the Vivado numbers mean.

1. The project in one minute

> Most serial controllers (and my v1) are fixed UART, SPI and I2C engines behind an AXI-APB bridge, and the CPU moves every byte through registers. v2 changes the controller itself. Protocols run as **programs** on a small serial engine, a **pin crossbar** lets any protocol use any connector while the board runs, and a **2 x 3 network-on-chip** connects the engines, a **sensor hub** and a **DMA writer**, so sensor data reaches memory and protocols talk to each other **without the CPU**. AXI-APB is kept only for set-up. The CPU is interrupted once per 32 sensor readings instead of twice per reading.

2. What the project is trying to prove

Not "less power", not "less area" and not a faster algorithm. v2 is **bigger** than v1 on purpose. The claim is that the extra hardware buys **flexibility** and **autonomy** (the CPU is no longer in the data path), and that this can be measured. The six claims and where each one is shown:

#	claim	v1	v2 (result)	evidence
1	a new protocol is a program, not a new bitstream	new engine, new bitstream (synthesis + implementation, 10-30 min)	UART, SPI, I2C, Manchester+CRC and the OLED driver are programs of 6-32 instructions, loaded in microseconds	`tb_se_programs` (69 checks): the UART, SPI and I2C programs give **bit-for-bit the same pin waveforms as the v1 engines**
2	pins can change role while the board runs	pins fixed when the design is built	any engine on JA/JB/JC/JD/OLED; a switch **never cuts a frame**; 2 clocks (20 ns) from an idle engine	`tb_xbar` (1.2 million checks), `tb_ssc2_system` [5]
3	the CPU no longer moves the data	10 register accesses + 2 interrupts **per reading** (about 13 accesses with the C driver's interrupt handler)	0 accesses + **1 interrupt per 32 readings** (= **64 x fewer interrupts**; about 0.1 accesses per reading including the interrupt handler)	`tb_ssc2_system` [3], [7]
4	any protocol can talk to any other, in hardware	one fixed path, through the CPU	PC (UART) -> network -> SPI flash -> back: `01 9F 03` -> `20 BA 19`, no CPU at all (in Way 1 there is no ARM)	`tb_ssc2_system` [2], `tb_zed2_standalone` [2], `tb_zed2_gate` [1]
5	every reading has a time stamp	none	each 16-byte record carries a microsecond time stamp and a sequence number	`tb_hub`, `tb_ssc2_system` [3]
6	an honest answer to "does a NoC pay off?"	-	bus vs crossbar vs mesh at 4, 8, 16 nodes: the mesh **loses on area and latency** at these sizes, matches the crossbar's throughput, and its logic depth does not grow with N	`noc_study/results.md`

"Time" improvements, if your mentor asks about complexity

- **CPU work per reading.** For N readings v1 costs N x (10 accesses + 2 interrupts); v2 costs about N/32 interrupts and 0.1 x N accesses. Both are O(N), but the constant is 64 x smaller for interrupts and about 100 x smaller for register accesses.
- **Changing a protocol:** microseconds (load 32 words) instead of a new synthesis and bitstream.
- **Switching a pin's role:** 20 ns from an idle engine.
- **Protocol conversion latency** is a fixed number of clocks with no software in the loop: 2 clocks (20 ns) per router hop; a 3-byte reply crosses the network in 15 clocks (150 ns) from start to last byte.

Power, honestly

v2 adds logic, so the FPGA part uses a little more power than v1, not less. Vivado's estimate for Way 1 is **0.234 W total on-chip**, mostly the device's static power and the clock tree (an estimate with default toggle rates, not a measurement). The possible saving is at **system** level: the ARM is woken 0.31 times per second instead of 20 times per second (at 10 readings per

second), so it can sleep. That saving has not been measured yet; it is a board measurement (for example a power meter on the 12 V input, with the ARM idle in v1 and v2 modes).

3. What got better and what it costs

	v1	**v2**
adding a protocol	new RTL + bitstream	load a program
pin roles	fixed at build time	change at runtime, never mid-frame
CPU work per reading	10 accesses + 2 IRQs	0 accesses + 1/32 IRQ
data path	byte by byte through APB	NoC + DMA into DDR (Way 2) / block RAM (Way 1)
protocol conversion	one fixed pair, through the CPU	any node to any node, in parallel
timing information	none	time stamp on every record
adding a channel	change the bus and the decoder	add a node and a router
area	about 2.3k LUTs	Way 1: **9 775 LUTs (18 %), 5 326 FFs (5 %)** in Vivado (first run); the full controller about 21k LUTs in the per-module yosys estimate (Vivado trims and maps better)
top SPI speed of the programmable engine	-	lower than the fixed engine (4 ticks per bit), so the fixed engines are kept
latency per network hop	-	2 clocks
verification effort	small	14 unit + 3 system testbenches, gate-level and equivalence checks

The costs are the ones slide 22 announced. The six routers are the largest single block.

4. How the output was verified

Every testbench checks its own results and ends with `ALL n CHECKS PASSED` (or `m OF n CHECKS FAILED`), so nobody has to read waveforms to know the answer. The reference ("golden") answers come from outside the design:

method	used for
v1 engines as reference models: the same bytes through a v1 engine and through a v2 program must give the same pin waveforms	serial-engine programs
device models with known answers: the Micron flash of the PmodSF3 returns the JEDEC ID `20 BA 19`, the TMP2 returns `0C 80` (25.0 C)	SPI, I2C, the whole system
scoreboards: every packet must arrive once, unchanged, in order, on the right port; no deadlock	NoC routers and mesh
AXI memory model that also checks the AXI protocol (bursts, 4 KB boundaries, handshakes)	DMA writer, whole system
random stress with random timing (1.2 million crossbar checks, thousands of packets)	crossbar, NoC, packet helpers
gate-level simulation: `tb_zed2_gate` drives only the board pins, so it also runs on the netlist after synthesis and after implementation	the Way 1 bitstream design
equivalence checks (timing fix): the new router against the old one clock by clock; formal proofs for the serial engine and DMA writer changes	see TIMING.md

Results (all passing): 14 unit testbenches (including `tb_xbar` 1 217 806 checks, `tb_noc_router` 1547, `tb_dma_writer` 242, `tb_se_engine` 257, `tb_hub` 197, `tb_se_programs` 69), `tb_ssc2_system` 36 checks, `tb_zed2_standalone` 8, `tb_zed2_gate` 11. How to run them and what each one proves: VERIFICATION.md.

On the board the same known answers are used: the PC sends `01 9F 03` and must get `20 BA 19`; the temperature must read `0C 80` or near it (your room temperature); the LEDs count records and batches at fixed rates; the logic analyser shows 8.68 us per UART bit at 115 200 baud.

5. The Vivado results (first run, Way 1)

item	value	comment
LUTs	9 775 of 53 200 (18.4 %)	v1 was about 2.3k: the price of flexibility
flip-flops	5 326 of 106 400 (5.0 %)	
block RAM	0	in Way 1 nobody reads the record RAM (its read port is not connected), so Vivado removes it; the DMA writer still writes and counts the records (LEDs)
power	0.234 W (estimate)	see section 2
timing	**WNS -0.687 ns** (failed)	the NoC router's path was too long for 100 MHz; **fixed**, see below
DRC ZPS7-1	warning	Way 1 does not use the ARM side; harmless for a JTAG-loaded design

The timing failure and its fix (details: TIMING.md). The router read its FIFO memory, looked up the route, arbitrated and drove the output register in one clock. Now each input keeps its front flit and its route in registers (worked out when the flit arrives), and the arbiters use one-hot vectors, so no memory read, table lookup or comparator sits in the critical path. Two other long paths (serial engine and DMA writer) were shortened too. The behaviour is unchanged to the

clock: the new router was compared with the old one cycle by cycle on millions of random clocks, and the other two changes were proven equivalent formally. After re-running implementation, write the new WNS (expected >= 0), LUT and FF numbers in MEASUREMENTS.md.

6. What is still to do on the board

The simulation results above are the reference. The slide-24 board numbers (baud rate, I2C frequency, top SPI clock, switch time, CPU load, area, Vivado Fmax of the NoC study) are taken with ZEDBOARD_GUIDE.md and MEASUREMENTS.md.

7. Questions your mentor may ask

"Isn't this still an APB peripheral?" No. APB is used only to set it up. Data moves on the network and through the DMA writer; in Way 1 there is no CPU at all and it still bridges the PC to the flash and stores sensor records.

"Why a network-on-chip for only 6 nodes?" At this size a crossbar would be smaller and faster; the study shows that and we report it. The NoC is chosen for modularity: a new engine is a new node with the same interface, and nothing central has to be redesigned. Its logic depth stays flat as nodes are added, which is where it starts to pay off (around 16 nodes).

"Is it faster than v1?" Per byte on the wire, no: the fixed engines are kept for top speed. It is faster where the CPU used to be in the loop: conversion, switching and data collection need no software.

"Did it meet timing?" The first run missed 100 MHz by 0.687 ns in the router. The router (and two other long paths) were restructured without changing behaviour, and the build now also runs physical optimisation. (Then give your new WNS.)

"How do you know the outputs are correct?" Every testbench compares against something independent: the v1 engines, device models with known answers (flash ID, temperature), scoreboards, an AXI checker. The board checks use the same known answers.

"What is new compared with the RP2040 PIO?" The PIO is general-purpose and locked inside one microcontroller. Here the engine has serial-bus instructions (open drain, CRC, side-set), it is reusable RTL, and it is combined with a crossbar, a network, a sensor hub and DMA.